

**Министерство науки и высшего образования Российской Федерации
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО**

Факультет безопасности информационных технологий

Дисциплина:

«Технологии и методы программирования»

ОТЧЕТ ПО ЛАБОРАТОРНОЙ РАБОТЕ №4

Выполнили:

Бардышев Артём Антонович,
студент группы N3346

(подпись)

Проверил:

Ищенко Алексей Петрович,
преподаватель, ФБИТ

(отметка о выполнении)

(подпись)

Санкт-Петербург

2025 г.

СОДЕРЖАНИЕ

Введение.....	4
1 РЕВЕРС ИСПОЛНЯЕМОГО ФАЙЛА.....	5
1.1 Первичное исследование.....	5
1.2 Анализ исполняемого файла (реверс).....	5
1.2.1 Динамический анализ и формирование гипотезы (x32dbg).....	5
1.2.2 Статический анализ и поиск функции проверки (IDA Pro)	6
1.2.3 Локализация функции-обработчика.....	6
1.2.4 Анализ алгоритма шифрования.....	6
1.3 Восстановление пароля	7
1.4 Проверка результата.....	8
Заключение.....	9
Список использованных источников	Ошибка! Закладка не определена.

ВВЕДЕНИЕ

Цель работы – Изучение методов реверс-инжиниринга программного обеспечения, анализ защитных механизмов и освоение инструментов для исследования исполняемых файлов. Получение практических навыков работы с дизассемблерами, декомпиляторами и инструментами статического анализа.

Реверс-инжиниринг

Реверс-инжиниринг (reverse engineering) — это процесс анализа программного обеспечения с целью понимания его структуры, алгоритмов работы и выявления уязвимостей или защитных механизмов.

Основные методы анализа

1. Статический анализ:

- Анализ кода без его выполнения
- Исследование структуры файла
- Декомпиляция и дизассемблирование
- Анализ строк и констант

2. Динамический анализ:

- Анализ программы во время выполнения
- Отладка (debugging)
- Трассировка выполнения
- Мониторинг системных вызовов

3. Гибридный анализ:

- Комбинация статического и динамического методов
- Символическое выполнение
- Фаззинг (fuzzing)

Защитные механизмы

Современные программы используют различные механизмы защиты от реверс-инжиниринга:

- Обфускация кода — усложнение читаемости кода
- Антиотладочные техники — обнаружение отладчиков
- Упаковщики (packers) — сжатие и шифрование кода
- Виртуальные машины — выполнение кода в виртуальной среде
- Контроль целостности — проверка изменений в памяти
- Антивирусные эвристики — обнаружение подозрительного поведения

1 РЕВЕРС ИСПОЛНЯЕМОГО ФАЙЛА

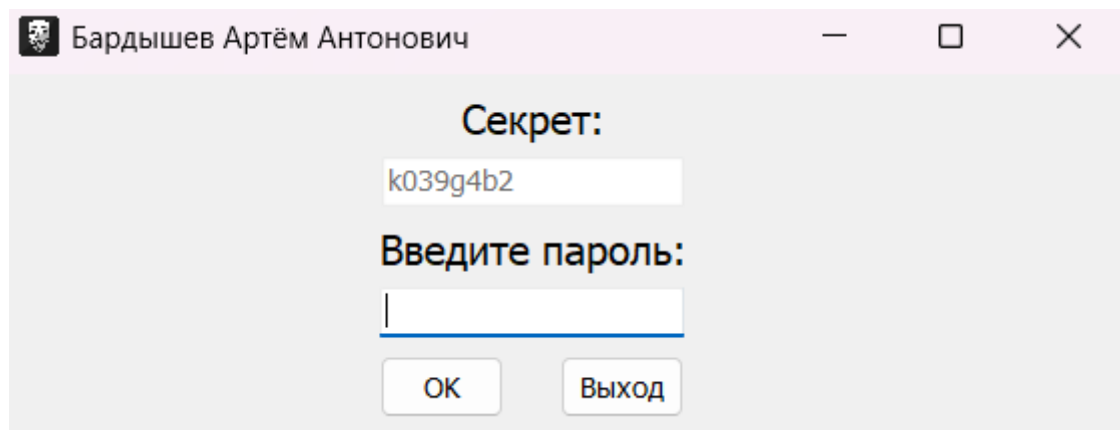
1.1 Первичное исследование

Так как контроль учетных записей (UAC) в ОС Windows блокирует исполнение файла LabReverse.exe, все дальнейшие операции с файлом выполнялись в изолированной среде «Песочница Windows».

Первичное исследование поведения программы показало следующее:

- При запуске отображается строка-подсказка: Секрет: k039g4b2.
- Программа требует ввода пароля длиной ровно 8 символов. При любой другой длине выводится сообщение об ошибке.
- После двух неудачных попыток ввода пароля правильной длины программа временно блокирует возможность ввода на 10 секунд.

Эти наблюдения позволили сделать вывод, что "секрет" является ключом или частью алгоритма, а искомый пароль имеет фиксированную 8-символьную длину.



1.2 Анализ исполняемого файла (реверс)

Анализ проводился в два этапа: первичный динамический анализ для формирования гипотез и последующий углубленный статический анализ для их подтверждения и восстановления алгоритма.

1.2.1 Динамический анализ и формирование гипотезы (x32dbg)

Первоначальное исследование проводилось с использованием отладчика x32dbg. Путем установки аппаратных точек останова на строку-секрет k039g4b2 было замечено, что обращения к этим данным происходят на ранних этапах запуска программы – после ее инициализации, но еще до отображения главного окна. Это позволило выдвинуть

гипотезу: программа предварительно вычисляет правильный пароль или его зашифрованный эталон при старте, а не в момент нажатия кнопки.

Дальнейший анализ памяти в отладчике позволил обнаружить рядом со строкой секретом еще одну ключевую строку данных. Учитывая ее длину (8 символов) и случайный набор символов, было сделано предположение, что это и есть заранее вычисленный зашифрованный эталон пароля.

1.2.2 Статический анализ и поиск функции проверки (IDA Pro)

Для подтверждения гипотезы и детального изучения алгоритма проводился статический анализ с помощью дизассемблера IDA Pro.

С помощью просмотра всех строковых литералов в файле (View -> Open subviews -> Strings) были повторно идентифицированы ранее обнаруженные строки, что подтвердило их статическое размещение в секции .data программы.

1.2.3 Локализация функции-обработчика

Для определения кода, отвечающего непосредственно за проверку пароля, был выполнен анализ перекрестных ссылок. В секции .data была найдена таблица опубликованных методов VCL (Visual Component Library), которая связывает события пользовательского интерфейса с функциями-обработчиками.

Событие Button1Click (нажатие на кнопку проверки) напрямую связано с адресом функции sub_40330C. Это однозначно идентифицирует ее как основную функцию, отвечающую за логику проверки пароля при взаимодействии с пользователем.

1.2.4 Анализ алгоритма шифрования

С помощью декомпилятора Hex-Rays (клавиша F5 в IDA Pro) был получен C подобный псевдокод функции sub_40330C. Детальный анализ этого кода позволил полностью восстановить алгоритм.

Алгоритм выполняет следующие действия:

1) Проверка длины пароля:

В самом начале функция получает введенный пользователем текст и проверяет его длину. Если длина не равна 8, выводится сообщение «Неверная длинна пароля».

2) Если Цикл шифрования: длина корректна, программа входит в основной цикл for ($j = 1; j < 9; ++j$), который выполняется 8 раз (Рисунок 4 –). Внутри цикла для каждого j -

го символа введенного пароля и j-го символа секрета (k039g4b2) происходят следующие преобразования:

а) Преобразование символов в числовые значения.

Программа берет ASCII-код символа пароля и символа секрета, после чего из каждого кода вычитается константа 48 (ASCII-код символа 0).

б) Побитовая операция XOR. Полученные числовые значения преобразуются в двоичное представление, после чего над ними выполняется побитовая операция XOR. Декомпилятор представляет эту 7 операцию в виде сложной логической конструкции, которая фактически реализует XOR для каждого бита.

с) Сборка результата.

Результат XOR (новая двоичная строка) преобразуется обратно в десятичное число, которое является ASCII-кодом символа зашифрованной строки. Этот символ добавляется к итоговому буферу unk_65D220.

2) Финальное сравнение:

После завершения цикла, полученная 8-символьная зашифрованная строка (unk_65D220) сравнивается с эталоном, хранящимся в глобальной переменной unk_669638.

1.3 Восстановление пароля

На основе проведенного анализа алгоритма шифрования была выведена математическая формула для восстановления исходного пароля.

Формула шифрования, применяемая программой: $\text{Код_Символа_Эталона} = (\text{Код_Символа_Пароля} - 48) \wedge (\text{Код_Символа_Секрета} - 48)$

Поскольку побитовая операция XOR ($\wedge$) обратима сама к себе (если $A \wedge B = C$, то $A = C \wedge B$), можно выразить из этой формулы код символа исходного пароля:

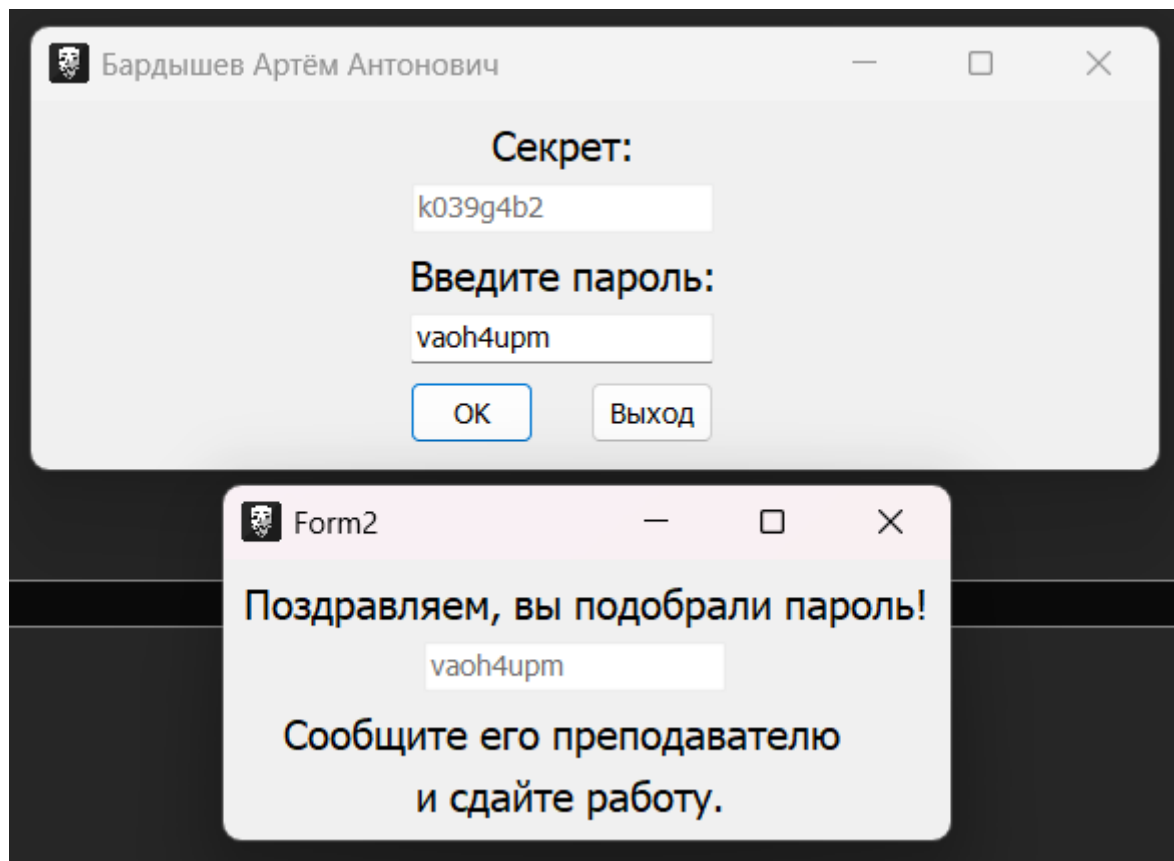
Формула для расшифровки: $\text{Код_Символа_Пароля} = (\text{Код_Символа_Эталона} \wedge (\text{Код_Символа_Секрета} - 48)) + 48$

Для автоматизации вычислений по данной формуле был разработан Python-скрипт, использующий известные нам данные.

Таким образом, искомый пароль был успешно восстановлен и равен vaoh4upm.

1.4 Проверка результата

Программа успешно приняла пароль, что подтверждает правильность восстановленного алгоритма.



ЗАКЛЮЧЕНИЕ

В ходе выполнения лабораторной работы были изучены:

Теоретические аспекты

1. Методы реверс-инжиниринга:

- Статический анализ — изучение кода без выполнения
- Динамический анализ — исследование во время выполнения
- Гибридные методы — комбинация подходов

2. Структура исполняемых файлов:

- Форматы PE, ELF, Mach-O
- Организация секций
- Таблицы импорта/экспорта
- Метаданные и ресурсы

3. Защитные механизмы:

- Упаковщики и обфускация
- Антиотладочные техники
- Контроль целостности
- Виртуальные машины

Практические навыки

1. Работа с инструментами:

- Использование дизассемблеров
- Применение декомпиляторов
- Анализ структуры файлов
- Извлечение данных

2. Анализ защищенных файлов:

- Обнаружение упаковщиков
- Распаковка файлов
- Обход простых защит
- Анализ обфусцированного кода

3. Исследование кода:

- Дизассемблирование
- Восстановление логики
- Анализ алгоритмов
- Поиск уязвимостей